



5

6

1080 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

Exception Handling

د. غاندي هسام



RB Informatics; 6/11/2025

برمجة ٣

Exception

- تعريف الـ Exception (الاستثناء) :
- هو حدث غير طبيعي يوقف التنفيذ العادي للبرنامج وهو يحدث اثناء تشغيل البرنامج (Runtime) بعد ان يكون قد تم تجميعه او تفسيره بنجاح .
- The Exception Handler (معالج الاستثناء) :
- هو مجموعة من الأوامر يتم تحديدها مسبقا تكون وظيفتها الوحيدة هي الاستجابة للاستثناء عند وقوعه وعدم السماح للبرنامج بالانهيار بدلا من التوقف وإظهار رسالة خطأ للمستخدم يقوم المستخدم بإضافة كود لمعالجة ذلك .
- لماذا نحتاج معالج الاستثناء Exception :
- قبل ظهور مفهوم معالجة الاستثناءات الحديث كان المبرمجون يستخدمون تقنية رموز الأخطاء القديمة في الطريقة القديمة اليدوية كانت كل دالة ترجع رقم بعد الانتهاء اذا كان الرقم صفر معنى ذلك ان العملية نجحت أي قيمة تظهر غير الصفر تشير الى نوع محدد من الأخطاء .
- كان على المبرمج ان يكتب يدويا سطرًا للتحقق من هذا الرقم في كل مرة يتم فيها استدعاء الدالة هذا يؤدي الى كود مزدحم وغير مقروء حيث يتداخل الجزء من الكود المتعلق بمنطق العمل مع الجزء المتعلق بمعالجة الأخطاء اما الطريقة الحديثة تسمح لك معالجة الاستثناءات بفصل الجزء المتعلق بمنطق العمل عن الجزء المتعلق بمعالجة الأخطاء بذلك نحصل على كود انظف واسهل للقراءة واقل عرضة للأخطاء
- تعرف لغة جافا استثناءات قياسية للأخطاء الشائعة مثل :
- القسمة على صفر , الملف غير موجود , تجاوز حدود المصفوفة .

"You're **not** crashing

you're just handling unexpected **exceptions** like a queen "



The Exception Hierarchy (هيكلية الاستثناءات) :

1. الوراثة هي المبدأ الأساسي الذي ينظم جميع الكلاسات الخاصة بالاستثناءات .
2. الاستثناء ليس مجرد حدث عشوائي بل هو كائن يتم إنشاؤه عند وقوع مشكلة يتم تحديد نوع هذا الكائن من خلال كلاس خاص به .
3. الكلاس `java.lang.Throwable` هو الكلاس الأب لجميع الاستثناءات التي يمكن رؤيتها او التقاطها بواسطة بيئة جافا الافتراضية.
4. عندما تكتشف الة جافا الافتراضية خطأ تقوم بإنشاء كائن جديد من الكلاس المناسب ثم ترمي هذا الكائن .
5. هذا الكائن الذي تم إنشاؤه هو الذي يحتوي على حالة الخطأ التفصيلية ويتم تمريره عبر سلسلة استدعاءات (call stack) للبحث عن معالج استثناء مناسب
6. في سلسلة الاستدعاءات call stack:
- إذا وجد catch ينفذه ويعطي الاستثناء المناسب
- إذا لم يجد catch ينزل الى ال JVM وبعدها يتسبب انهيار البرنامج crash

الكلاسات الفرعية لكلاس Throwable :

- **كلاس Exception :** هي أخطاء قابلة للاسترداد أي يمكن معالجتها ويستمر عمل البرنامج بشكل طبيعي تتعلق بنشاط البرنامج ممكن ان تكون أخطاء منطقية , أخطاء تتعلق بعمليات الدخال والإخراج , سوء استخدام بنى البيانات , محاولة اجراء عمليات غير صحيحة حسابياالخ
- التعامل مع الاستثناءات يؤدي الى برامج اكثر متانة واستقرار
- **كلاس Error :** يشير عادة الى مشكلة خطيرة تحدث وتكون خارج سيطرة البرنامج ولا يمكن للبرنامج ان يتعامل معها بشكل طبيعي ويتوقف بشكل كامل وفوري غالبا ما يشير الى مشاكل خارجية عن الكود مثل مشاكل في الذاكرة او النظام الأساسي (stack overflow , out of memory, virtual machine)
- **مفهوم fail fast :**
- لغة جافا تعتمد على هذا المفهوم والذي يقولافشل بسرعة بدل ان تجعل المشكلة تكبر
- أي ضع استثناء فشل في أي حالة يمكن ان يفشل فيها البرنامج بدل من الوقوع بمشاكل اكبر مثل استثناء القسمة على صفر `ArithmeticException` ففورا عندما يجد قسمة على صفر يرمي البرنامج هذا الاستثناء قبل ان يصل الى مرحلة الانهيار ولهذا السبب تم انشاء فلسفة الاستثناءات

اساسيات التعامل مع الاستثناءات :

٤ تتم إدارة معالجة استثناءات في جافا من خلال خمس كلمات مفتاحية

- Try: لتحديد كتلة الكود التي قد تتسبب في حدوث استثناء .
- Catch: لالتقاط الاستثناء (اذا حدث) وتنفيذ كود معالجة.
- throw : لرمي استثناء صريح يدوي داخل الكود .
- Throws : للاعلان عن ان الدالة ترمي نوع معين من الاستثناءات .
- Finally : لتحديد كتلة كود يجب تنفيذها دائما سواء حدث استثناء ام لم يحدث .
- اذا حدث استثناء داخل كتلة try يتم رميه ويتوقف التنفيذ العادي لكتلة try ثم يبدأ نظام جافا بالبحث عن كتلة catch مناسبة لالتقاط هذا النوع من الكائن الكود داخل catch يجب ان يهدف الى ابلاغ المستخدم بالخطأ اذا لم يحدث استثناء يتم تخطي كتلة catch.
- مفهوم Narrow first , Board last :
- يقول هذا المفهوم اننا يجب ان نرتب ال catch من الاكثر تحديدا الى الاكثر عموما مثل

```
catch (NumberFormatException e) { }
catch (RuntimeException e)
{ }
catch (Exception e)
{ }
```

الشكل العام ل try and catch :

```
try{
//block of code to monitor for errors
}
catch(ExcepType1 exOb ){
//handler for ExcepType1
}
catch (ExcepType2 exOb){
//handler for ExcepType1
}
.
.
.
```

// Demonstrate exception handling.

```
class ExcDemo1 {
    public static void main(String[] args) {
        int[] nums = new int[4];
        try { ← Create a try block
            System.out.println("Before exception is generated.");
            // Generate an index-out-of-bounds exception.
            nums[7] = 10; ← Attempt to index past nums boundary.
            System.out.println("this won't be displayed");
        }
        catch (ArrayIndexOutOfBoundsException exc) { ← catch the exception

            System.out.println("Index out-of-bounds!");
        }
        System.out.println("After catch statement.");
    }
}
```

Output

```
Before exception is generated.
Index out-of-bounds!
After catch statement.
```

■ شرح الكود :

- يتم انشاء مصفوفة حجمها 4 ثم يتم تنفيذ الكود داخل كتلة try عند وصول التنفيذ الى السطر المسبب للاستثناء : محاولة الوصول لفهرس 7 في مصفوفة حجمها 4 يتوقف تنفيذ كتلة try ثم ينتقل الى كتلة catch المطابقة لنوع الاستثناء ويتم تنفيذ كود المعالجة الخاص بها بعد انتهاء كتلة catch يستأنف البرنامج التنفيذ العادي من اول سطر يلي الكتلة المغلقة .



ملاحظة هامة جدا :

الاستثناءات ليس من الضرورة ان تحدث بشكل مباشر داخل كتلة try يمكن ل try ان يستدعي دالة واذا حدث الاستثناء داخل تلك الدالة فانه يعود صعودا الى كتلة try الاصلية للتقاطه

مثال عن الملاحظة :

```
class ExcTest {  
    // Generate an exception.  
    static void genException() {  
        int[] nums = new int[4];  
  
        System.out.println("Before exception is generated");  
  
        // generate an index out-of-bounds exception  
        nums[7] = 10; ← Exception generated here  
        System.out.println("this won't be displayed");  
    }  
}  
  
class ExcDemo2 {  
    public static void main(String[] args) {  
        try {  
            ExcTest.genException();  
        } catch (ArrayIndexOutOfBoundsException exc) {  
            // catch the exception  
            System.out.println("Index out-of-bounds!");  
        }  
        System.out.println("After catch statement.");  
    }  
}
```

Output

```
Before exception is generated  
Index out-of-bounds!  
After catch statement.
```



■ شرح الكود:

■ كلاس ExcTest :

يتم تعريف دالة genException() مهمتها توليد الاستثناء
 انشاء مصفوفة اعداد صحيحة بحجم 4 عناصر (الفهارس من 0 الى 3) محاولة الوصول الى الفهرس 7
 (خارج الحدود) مما يؤدي الى رمي استثناء ArrayIndexOutOfBoundsException تلقائيا
 يتوقف التنفيذ في الدالة فور رمي الاستثناء .

■ كلاس ExcDemo2:

■ كتلة try أي خطأ يحدث داخل هذه الكتلة او الدوال الذي يستدعيها سيتم التقاطه
 ■ استدعاء الدالة المسببة للخطأ يبدأ تنفيذها وحالما ترمي الاستثناء تلتقطه كتلة catch ويتم تنفيذ خطة
 المعالجة :طباعة رسالة للمستخدم بدلا من ترك البرنامج يتوقف
 ■ ثم يتم استئناف التنفيذ بعد انتهاء المعالجة

■ مفهوم Don't catch what you can't handle :

■ يقول هذا المفهوم اذا كنت لا تعرف ماذا ستفعل بالاستثناء وماذا ستخرج منه وما الهدف منه فلا
 تلتقط هذا الاستثناء لأنه سيكون عبء ومشكلة اكثر من ان يكون حل حينها
 ■ اسأل نفسك اسئلة قبل كتابة اي catch:
 1. هل استطيع اصلاح الخطأ؟
 2. هل يحتاج المستخدم ان يعرف ان هذا خطأ؟
 اذا كان الجواب لا فالاستثناء غير مفيد حينها

```
try {
    chaseYourDreams();
} catch (JavaException e) {
    // You don't give up.
    keepGoing();
}
```

